```gs
/**
 * 2nd Year UNAM - Extraordinary Exam Study Guide App
 * Code.gs
 *
 * Replacement backend with live tracking.
 *
 * It records:
 * - student start time as soon as the student clicks Start
 * - current question and progress
 * - each answer as soon as it is checked
 * - time spent on each question
 * - browser/app focus changes
 * - total time outside the app
 * - final score at submission
 */

const SPREADSHEET_ID = '1J5S0Pz8QrJSy7TA7p1vzOX2PoPxGzKlysmBDEvv-MBM';
const EXAM_VERSION = '2nd Year UNAM - Second Final Exam Study Guide';
const LIVE_SHEET_NAME = 'Live Monitor';

function doGet(e) {
  return HtmlService
    .createHtmlOutputFromFile('Index')
    .setTitle('2nd Year UNAM - Second Final Exam Study Guide')
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

/**
 * Call this immediately when the student clicks Start Practice.
 *
 * Expected payload:
 * {
 *   studentName: string,
 *   totalQuestions: number
 * }
 */
```

```
function startLiveAttempt(payload) {
  return withLock_(function() {
    payload = payload || {};

    const studentName = cleanStudentName_(payload.studentName);
    if (!studentName) throw new Error('Student name is required.');

    const totalQuestions = Number(payload.totalQuestions || 0);
    const now = new Date();
    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheetName = makeSafeSheetName_(studentName);
    const sheet = getOrCreateSheet_(ss, sheetName);

    sheet.clear();
    setupStudentSheet_(sheet);

    const attemptId = Utilities.getUuid();
    const startedAt = formatDateTime_(now);

    const summaryRows = [
      ['Student Name', studentName],
      ['Status', 'In progress'],
      ['Started At', startedAt],
      ['Last Seen', startedAt],
      ['Current Question', 'Not opened yet'],
      ['Progress', '0 / ' + totalQuestions + ' (0%)'],
      ['Progress Percent', 0],
      ['Score So Far', '0 / ' + totalQuestions],
      ['Total Questions', totalQuestions],
      ['Total Time', '0 sec'],
      ['Total Time Seconds', 0],
      ['Total Time Outside App', '0 sec'],
      ['Total Time Outside App Seconds', 0],
      ['Currently Outside App?', 'No'],
      ['Last Outside Started At', ''],
      ['Attempt ID', attemptId],
```

```
      ['Exam Version', EXAM_VERSION]
    ];

    sheet.getRange(1, 1, summaryRows.length, 2).setValues(summaryRows);
    applySummaryFormatting_(sheet, summaryRows.length);

    getOrCreateLiveMonitor_(ss);
    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheetName,
      status: 'In progress',
      startedAt: startedAt,
      lastSeen: startedAt,
      currentQuestion: 'Not opened yet',
      progress: '0 / ' + totalQuestions + ' (0%)',
      score: '0 / ' + totalQuestions,
      totalTime: '0 sec',
      outsideTime: '0 sec',
      outsideNow: 'No'
    });

    appendEvent_(sheet, now, 'START', 'Student clicked Start Practice.');

    return {
      status: 'success',
      attemptId: attemptId,
      sheetName: sheetName,
      startedAt: startedAt
    };
  });
}

/**
 * Call this every time a new question is rendered.
 *
 * Expected payload:
```

```
 * {
 *   studentName: string,
 *   attemptId: string,
 *   questionNumber: number,
 *   topic: string,
 *   section: string,
 *   questionText: string,
 *   currentIndex: number, // 1-based
 *   totalQuestions: number,
 *   score: number
 * }
 */
function updateLiveQuestion(payload) {
  return withLock_(function() {
    payload = payload || {};

    const studentName = cleanStudentName_(payload.studentName);
    if (!studentName) throw new Error('Student name is required.');

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = getOrCreateSheet_(ss, makeSafeSheetName_(studentName));
    const now = new Date();

    const totalQuestions = Number(payload.totalQuestions ||
getSummaryValue_(sheet, 'Total Questions') || 0);
    const currentIndex = Number(payload.currentIndex || payload.questionNumber
|| 0);
    const score = Number(payload.score || 0);
    const progressPercent = totalQuestions > 0 ? Math.round((currentIndex /
totalQuestions) * 100) : 0;
    const currentQuestion = 'Q' + (payload.questionNumber || currentIndex) + '
- ' + (payload.topic || '');

    setSummaryValue_(sheet, 'Status', 'In progress');
    setSummaryValue_(sheet, 'Last Seen', formatDateTime_(now));
    setSummaryValue_(sheet, 'Current Question', currentQuestion);
```

```
    setSummaryValue_(sheet, 'Progress', currentIndex + ' / ' + totalQuestions
+ ' (' + progressPercent + '%)');
    setSummaryValue_(sheet, 'Progress Percent', progressPercent);
    setSummaryValue_(sheet, 'Score So Far', score + ' / ' + totalQuestions);
    updateTotalTime_(sheet, now);

    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheet.getName(),
      status: 'In progress',
      startedAt: getSummaryValue_(sheet, 'Started At'),
      lastSeen: formatDateTime_(now),
      currentQuestion: currentQuestion,
      progress: currentIndex + ' / ' + totalQuestions + ' (' + progressPercent
+ '%)',
      score: score + ' / ' + totalQuestions,
      totalTime: getSummaryValue_(sheet, 'Total Time'),
      outsideTime: getSummaryValue_(sheet, 'Total Time Outside App'),
      outsideNow: getSummaryValue_(sheet, 'Currently Outside App?')
    });

    appendEvent_(
      sheet,
      now,
      'QUESTION_OPENED',
      currentQuestion + ' | ' + (payload.questionText || '')
    );

    return { status: 'success' };
  });
}

/**
 * Call this as soon as the student clicks Check Answer.
 *
 * Expected payload:
```

```
 * {
 *   studentName: string,
 *   attemptId: string,
 *   questionNumber: number,
 *   topic: string,
 *   section: string,
 *   questionText: string,
 *   studentAnswer: string,
 *   correctAnswer: string,
 *   isCorrect: boolean,
 *   feedback: string,
 *   timeSpentSeconds: number,
 *   currentIndex: number,
 *   totalQuestions: number,
 *   score: number
 * }
 */
function recordLiveAnswer(payload) {
  return withLock_(function() {
    payload = payload || {};

    const studentName = cleanStudentName_(payload.studentName);
    if (!studentName) throw new Error('Student name is required.');

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = getOrCreateSheet_(ss, makeSafeSheetName_(studentName));
    const now = new Date();

    ensureAnswersHeader_(sheet);

    const totalQuestions = Number(payload.totalQuestions ||
getSummaryValue_(sheet, 'Total Questions') || 0);
    const currentIndex = Number(payload.currentIndex || payload.questionNumber
|| 0);
    const score = Number(payload.score || 0);
```

```
    const progressPercent = totalQuestions > 0 ? Math.round((currentIndex /
totalQuestions) * 100) : 0;
    const timeSpentSeconds = Number(payload.timeSpentSeconds || 0);

    const row = [
      payload.questionNumber || '',
      payload.section || '',
      payload.topic || '',
      payload.questionText || '',
      payload.studentAnswer || '',
      payload.correctAnswer || '',
      payload.isCorrect ? 'Yes' : 'No',
      payload.feedback || '',
      formatSeconds_(timeSpentSeconds),
      timeSpentSeconds,
      formatDateTime_(now)
    ];

    sheet.appendRow(row);

    const lastRow = sheet.getLastRow();
    sheet.getRange(lastRow, 1, 1, row.length)
      .setBackground(payload.isCorrect ? '#ecfdf5' : '#fef2f2');
    sheet.getRange(lastRow, 4, 1, 5).setWrap(true);

    setSummaryValue_(sheet, 'Last Seen', formatDateTime_(now));
    setSummaryValue_(sheet, 'Progress', currentIndex + ' / ' + totalQuestions
+ ' (' + progressPercent + '%)');
    setSummaryValue_(sheet, 'Progress Percent', progressPercent);
    setSummaryValue_(sheet, 'Score So Far', score + ' / ' + totalQuestions);
    updateTotalTime_(sheet, now);

    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheet.getName(),
      status: getSummaryValue_(sheet, 'Status') || 'In progress',
```

```
      startedAt: getSummaryValue_(sheet, 'Started At'),
      lastSeen: formatDateTime_(now),
      currentQuestion: getSummaryValue_(sheet, 'Current Question'),
      progress: getSummaryValue_(sheet, 'Progress'),
      score: score + ' / ' + totalQuestions,
      totalTime: getSummaryValue_(sheet, 'Total Time'),
      outsideTime: getSummaryValue_(sheet, 'Total Time Outside App'),
      outsideNow: getSummaryValue_(sheet, 'Currently Outside App?')
    });

    appendEvent_(
      sheet,
      now,
      'ANSWER_RECORDED',
      'Q' + payload.questionNumber + ' | ' + row[4] + ' | Correct: ' + row[6]
    );

    return { status: 'success' };
  });
}

/**
 * Call this when the tab loses/gains focus or document visibility changes.
 *
 * Expected payload:
 * {
 *   studentName: string,
 *   attemptId: string,
 *   eventType: "outside" | "inside",
 *   reason: "blur" | "focus" | "hidden" | "visible" | string
 * }
 */
function recordFocusEvent(payload) {
  return withLock_(function() {
    payload = payload || {};
```

```
    const studentName = cleanStudentName_(payload.studentName);
    if (!studentName) throw new Error('Student name is required.');

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = getOrCreateSheet_(ss, makeSafeSheetName_(studentName));
    const now = new Date();
    const eventType = String(payload.eventType || '').toLowerCase();
    const reason = payload.reason || '';

    if (eventType === 'outside') {
      if (getSummaryValue_(sheet, 'Currently Outside App?') !== 'Yes') {
        setSummaryValue_(sheet, 'Currently Outside App?', 'Yes');
        setSummaryValue_(sheet, 'Last Outside Started At',
formatDateTime_(now));
        appendEvent_(sheet, now, 'LEFT_APP', reason);
      }
    }

    if (eventType === 'inside') {
      const wasOutside = getSummaryValue_(sheet, 'Currently Outside App?') ===
'Yes';
      const outsideStartedAt = parseDateTime_(getSummaryValue_(sheet, 'Last
Outside Started At'));

      if (wasOutside && outsideStartedAt) {
        const addedSeconds = Math.max(0, Math.round((now - outsideStartedAt) /
1000));
        const previousSeconds = Number(getSummaryValue_(sheet, 'Total Time
Outside App Seconds') || 0);
        const totalOutsideSeconds = previousSeconds + addedSeconds;

        setSummaryValue_(sheet, 'Total Time Outside App Seconds',
totalOutsideSeconds);
        setSummaryValue_(sheet, 'Total Time Outside App',
formatSeconds_(totalOutsideSeconds));
```

```
        appendEvent_(sheet, now, 'RETURNED_TO_APP', reason + ' | Outside for '
+ formatSeconds_(addedSeconds));
      }

      setSummaryValue_(sheet, 'Currently Outside App?', 'No');
      setSummaryValue_(sheet, 'Last Outside Started At', '');
    }

    setSummaryValue_(sheet, 'Last Seen', formatDateTime_(now));
    updateTotalTime_(sheet, now);

    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheet.getName(),
      status: getSummaryValue_(sheet, 'Status') || 'In progress',
      startedAt: getSummaryValue_(sheet, 'Started At'),
      lastSeen: formatDateTime_(now),
      currentQuestion: getSummaryValue_(sheet, 'Current Question'),
      progress: getSummaryValue_(sheet, 'Progress'),
      score: getSummaryValue_(sheet, 'Score So Far'),
      totalTime: getSummaryValue_(sheet, 'Total Time'),
      outsideTime: getSummaryValue_(sheet, 'Total Time Outside App'),
      outsideNow: getSummaryValue_(sheet, 'Currently Outside App?')
    });

    return { status: 'success' };
  });
}

/**
 * Optional: call this every 15-30 seconds while the exam is open.
 */
function liveHeartbeat(payload) {
  return withLock_(function() {
    payload = payload || {};
```

```
    const studentName = cleanStudentName_(payload.studentName);
    if (!studentName) throw new Error('Student name is required.');

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = getOrCreateSheet_(ss, makeSafeSheetName_(studentName));
    const now = new Date();

    setSummaryValue_(sheet, 'Last Seen', formatDateTime_(now));
    updateTotalTime_(sheet, now);

    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheet.getName(),
      status: getSummaryValue_(sheet, 'Status') || 'In progress',
      startedAt: getSummaryValue_(sheet, 'Started At'),
      lastSeen: formatDateTime_(now),
      currentQuestion: getSummaryValue_(sheet, 'Current Question'),
      progress: getSummaryValue_(sheet, 'Progress'),
      score: getSummaryValue_(sheet, 'Score So Far'),
      totalTime: getSummaryValue_(sheet, 'Total Time'),
      outsideTime: getSummaryValue_(sheet, 'Total Time Outside App'),
      outsideNow: getSummaryValue_(sheet, 'Currently Outside App?')
    });

    return { status: 'success' };
  });
}

/**
 * Final save. This keeps compatibility with your current HTML saveResults().
 */
function recordStudyGuideAttempt(submissionData) {
  return withLock_(function() {
    if (!submissionData) throw new Error('No submission data was received.');

    const studentName = cleanStudentName_(submissionData.studentName);
```

```
    if (!studentName) throw new Error('Student name is required.');

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = getOrCreateSheet_(ss, makeSafeSheetName_(studentName));
    const now = new Date();

    const score = Number(submissionData.score || 0);
    const totalQuestions = Number(submissionData.totalQuestions || 0);
    const percentage = Number(submissionData.percentage || 0);
    const totalTimeSeconds = Number(submissionData.totalTimeSeconds || 0);
    const answers = Array.isArray(submissionData.answers) ?
submissionData.answers : [];

    if (sheet.getLastRow() < 23) {
      sheet.clear();
      setupStudentSheet_(sheet);
      sheet.getRange(1, 1, 17, 2).setValues([
        ['Student Name', studentName],
        ['Status', 'Submitted'],
        ['Started At', formatDateTime_(now)],
        ['Last Seen', formatDateTime_(now)],
        ['Current Question', 'Finished'],
        ['Progress', totalQuestions + ' / ' + totalQuestions + ' (100%)'],
        ['Progress Percent', 100],
        ['Score So Far', score + ' / ' + totalQuestions],
        ['Total Questions', totalQuestions],
        ['Total Time', formatSeconds_(totalTimeSeconds)],
        ['Total Time Seconds', totalTimeSeconds],
        ['Total Time Outside App', '0 sec'],
        ['Total Time Outside App Seconds', 0],
        ['Currently Outside App?', 'No'],
        ['Last Outside Started At', ''],
        ['Attempt ID', submissionData.attemptId || Utilities.getUuid()],
        ['Exam Version', EXAM_VERSION]
      ]);
      applySummaryFormatting_(sheet, 17);
```

```
    }

    setSummaryValue_(sheet, 'Status', 'Submitted');
    setSummaryValue_(sheet, 'Last Seen', formatDateTime_(now));
    setSummaryValue_(sheet, 'Current Question', 'Finished');
    setSummaryValue_(sheet, 'Progress', totalQuestions + ' / ' +
totalQuestions + ' (100%)');
    setSummaryValue_(sheet, 'Progress Percent', 100);
    setSummaryValue_(sheet, 'Score So Far', score + ' / ' + totalQuestions);
    setSummaryValue_(sheet, 'Final Score', score + ' / ' + totalQuestions);
    setSummaryValue_(sheet, 'Percentage', percentage + '%');
    setSummaryValue_(sheet, 'Total Time', formatSeconds_(totalTimeSeconds));
    setSummaryValue_(sheet, 'Total Time Seconds', totalTimeSeconds);
    setSummaryValue_(sheet, 'Currently Outside App?', 'No');
    setSummaryValue_(sheet, 'Last Outside Started At', '');

    ensureAnswersHeader_(sheet);

    const existingAnswerRows = Math.max(0, sheet.getLastRow() - 23);
    if (existingAnswerRows === 0 && answers.length > 0) {
      const rows = answers.map(function(item) {
        const timeSpentSeconds = Number(item.timeSpentSeconds || 0);
        return [
          item.questionNumber || '',
          item.section || '',
          item.topic || '',
          item.questionText || '',
          item.studentAnswer || '',
          item.correctAnswer || '',
          item.isCorrect ? 'Yes' : 'No',
          item.feedback || '',
          formatSeconds_(timeSpentSeconds),
          timeSpentSeconds,
          formatDateTime_(now)
        ];
      });
```

```
      sheet.getRange(24, 1, rows.length, 11).setValues(rows);
    }

    appendEvent_(sheet, now, 'SUBMITTED', 'Final score: ' + score + ' / ' +
totalQuestions + ' (' + percentage + '%)');

    upsertLiveMonitor_(ss, {
      studentName: studentName,
      sheetName: sheet.getName(),
      status: 'Submitted',
      startedAt: getSummaryValue_(sheet, 'Started At'),
      lastSeen: formatDateTime_(now),
      currentQuestion: 'Finished',
      progress: totalQuestions + ' / ' + totalQuestions + ' (100%)',
      score: score + ' / ' + totalQuestions,
      totalTime: formatSeconds_(totalTimeSeconds),
      outsideTime: getSummaryValue_(sheet, 'Total Time Outside App'),
      outsideNow: 'No'
    });

    return {
      status: 'success',
      message: 'Your study guide attempt has been recorded successfully.',
      sheetName: sheet.getName(),
      score: score,
      totalQuestions: totalQuestions,
      percentage: percentage
    };
  });
}

function setupStudentSheet_(sheet) {
  sheet.getRange('A1:B17').clearContent();
  sheet.getRange('A19:C19').setValues([['Live Progress Bar', '', '']]);
  sheet.getRange('A20:C20').setValues([['Progress', '', '']]);
```

```
sheet.getRange('A22:K22').setValues([[
  'Question #',
  'Section',
  'Topic',
  'Question',
  'Student Answer',
  'Correct Answer',
  'Correct?',
  'Feedback',
  'Time Spent',
  'Time Spent Seconds',
  'Recorded At'
]]);

sheet.getRange('A19:C19')
  .merge()
  .setFontWeight('bold')
  .setBackground('#1d4ed8')
  .setFontColor('#ffffff');

sheet.getRange('A20').setFontWeight('bold');
sheet.getRange('B20:C20').merge().setBackground('#e5e7eb');

sheet.getRange('A22:K22')
  .setFontWeight('bold')
  .setBackground('#0f766e')
  .setFontColor('#ffffff')
  .setHorizontalAlignment('center');

sheet.setFrozenRows(22);
sheet.setColumnWidth(1, 95);
sheet.setColumnWidth(2, 170);
sheet.setColumnWidth(3, 190);
sheet.setColumnWidth(4, 430);
sheet.setColumnWidth(5, 220);
sheet.setColumnWidth(6, 220);
```

```
  sheet.setColumnWidth(7, 95);
  sheet.setColumnWidth(8, 430);
  sheet.setColumnWidth(9, 130);
  sheet.setColumnWidth(10, 150);
  sheet.setColumnWidth(11, 170);
}

function ensureAnswersHeader_(sheet) {
  if (sheet.getRange('A22').getValue() === 'Question #') return;
  setupStudentSheet_(sheet);
}

function applySummaryFormatting_(sheet, rowCount) {
  sheet.getRange(1, 1, rowCount, 1)
    .setFontWeight('bold')
    .setBackground('#1d4ed8')
    .setFontColor('#ffffff');

  sheet.getRange(1, 2, rowCount, 1).setBackground('#eff6ff');
  sheet.getRange('A1:B17').setVerticalAlignment('middle');
  updateProgressBar_(sheet);
}

function updateProgressBar_(sheet) {
  const percent = Number(getSummaryValue_(sheet, 'Progress Percent') || 0);
  const blocks = Math.max(0, Math.min(20, Math.round(percent / 5)));
  const bar = repeat_('▓', blocks) + repeat_('░', 20 - blocks) + ' ' + percent
+ '%';

  sheet.getRange('B20').setValue(bar);
  sheet.getRange('B20:C20').setBackground(progressColor_(percent));
}

function updateTotalTime_(sheet, now) {
  const startedAt = parseDateTime_(getSummaryValue_(sheet, 'Started At'));
  if (!startedAt) return;
```

```
  const totalTimeSeconds = Math.max(0, Math.round((now - startedAt) / 1000));
  setSummaryValue_(sheet, 'Total Time Seconds', totalTimeSeconds, false);
  setSummaryValue_(sheet, 'Total Time', formatSeconds_(totalTimeSeconds),
false);
  updateProgressBar_(sheet);
}

function getOrCreateLiveMonitor_(ss) {
  let sheet = ss.getSheetByName(LIVE_SHEET_NAME);
  if (!sheet) sheet = ss.insertSheet(LIVE_SHEET_NAME, 0);

  if (sheet.getLastRow() === 0 || sheet.getRange('A1').getValue() !== 'Student
Name') {
    sheet.clear();
    sheet.getRange(1, 1, 1, 11).setValues([[
      'Student Name',
      'Student Sheet',
      'Status',
      'Started At',
      'Last Seen',
      'Current Question',
      'Progress',
      'Score',
      'Total Time',
      'Outside App Time',
      'Outside Now?'
    ]]);
    sheet.getRange('A1:K1')
      .setFontWeight('bold')
      .setBackground('#111827')
      .setFontColor('#ffffff');
    sheet.setFrozenRows(1);
    sheet.autoResizeColumns(1, 11);
  }
```

```
  return sheet;
}

function upsertLiveMonitor_(ss, data) {
  const sheet = getOrCreateLiveMonitor_(ss);
  const values = sheet.getDataRange().getValues();
  let rowIndex = -1;

  for (let i = 1; i < values.length; i++) {
    if (values[i][0] === data.studentName) {
      rowIndex = i + 1;
      break;
    }
  }

  const row = [
    data.studentName || '',
    data.sheetName || '',
    data.status || '',
    data.startedAt || '',
    data.lastSeen || '',
    data.currentQuestion || '',
    data.progress || '',
    data.score || '',
    data.totalTime || '',
    data.outsideTime || '',
    data.outsideNow || ''
  ];

  if (rowIndex === -1) {
    sheet.appendRow(row);
    rowIndex = sheet.getLastRow();
  } else {
    sheet.getRange(rowIndex, 1, 1, row.length).setValues([row]);
  }
```

```
  sheet.getRange(rowIndex, 1, 1, row.length)
    .setBackground(data.outsideNow === 'Yes' ? '#fef3c7' : '#ffffff');
}

function appendEvent_(sheet, date, eventType, details) {
  const eventStartCol = 13;

  if (sheet.getRange(1, eventStartCol).getValue() !== 'Event Time') {
    sheet.getRange(1, eventStartCol, 1, 3).setValues([['Event Time', 'Event
Type', 'Details']]);
    sheet.getRange(1, eventStartCol, 1, 3)
      .setFontWeight('bold')
      .setBackground('#7c3aed')
      .setFontColor('#ffffff');
    sheet.setColumnWidth(eventStartCol, 170);
    sheet.setColumnWidth(eventStartCol + 1, 160);
    sheet.setColumnWidth(eventStartCol + 2, 420);
  }

  const nextRow = Math.max(2, sheet.getRange(sheet.getMaxRows(),
eventStartCol).getNextDataCell(SpreadsheetApp.Direction.UP).getRow() + 1);
  sheet.getRange(nextRow, eventStartCol, 1, 3).setValues([[
    formatDateTime_(date),
    eventType,
    details || ''
  ]]);
  sheet.getRange(nextRow, eventStartCol + 2).setWrap(true);
}

function getSummaryValue_(sheet, label) {
  const values = sheet.getRange(1, 1, 21, 2).getValues();

  for (let i = 0; i < values.length; i++) {
    if (values[i][0] === label) return values[i][1];
  }
```

```
  return '';
}

function setSummaryValue_(sheet, label, value, refreshProgressBar) {
  if (refreshProgressBar === undefined) refreshProgressBar = true;

  const values = sheet.getRange(1, 1, Math.max(21, sheet.getLastRow()),
2).getValues();

  for (let i = 0; i < values.length; i++) {
    if (values[i][0] === label) {
      sheet.getRange(i + 1, 2).setValue(value);
      if (refreshProgressBar) updateProgressBar_(sheet);
      return;
    }
  }

  const row = Math.max(1, sheet.getLastRow() + 1);
  sheet.getRange(row, 1, 1, 2).setValues([[label, value]]);
  if (refreshProgressBar) updateProgressBar_(sheet);
}

function cleanStudentName_(name) {
  if (!name) return '';

  return String(name)
    .trim()
    .replace(/\s+/g, ' ');
}

function makeSafeSheetName_(name) {
  let safeName = String(name)
    .replace(/[\\\/\?\*\[\]]/g, '')
    .trim();

  if (!safeName) safeName = 'Unnamed Student';
```

```
  if (safeName.length > 90) safeName = safeName.substring(0, 90);

  return safeName;
}

function getOrCreateSheet_(ss, sheetName) {
  return ss.getSheetByName(sheetName) || ss.insertSheet(sheetName);
}

function formatDateTime_(date) {
  return Utilities.formatDate(
    date,
    Session.getScriptTimeZone(),
    'yyyy-MM-dd HH:mm:ss'
  );
}

function parseDateTime_(value) {
  if (!value) return null;
  if (Object.prototype.toString.call(value) === '[object Date]' &&
!isNaN(value)) return value;

  const parsed = new Date(String(value).replace(' ', 'T'));
  return isNaN(parsed.getTime()) ? null : parsed;
}

function formatSeconds_(seconds) {
  seconds = Number(seconds || 0);

  const minutes = Math.floor(seconds / 60);
  const remainingSeconds = seconds % 60;

  if (minutes <= 0) return remainingSeconds + ' sec';

  return minutes + ' min ' + remainingSeconds + ' sec';
}
```

```
function progressColor_(percent) {
  percent = Number(percent || 0);
  if (percent >= 100) return '#dcfce7';
  if (percent >= 67) return '#dbeafe';
  if (percent >= 34) return '#fef3c7';
  return '#fee2e2';
}

function repeat_(text, count) {
  let output = '';
  for (let i = 0; i < count; i++) output += text;
  return output;
}

function withLock_(callback) {
  const lock = LockService.getScriptLock();
  lock.waitLock(20000);

  try {
    return callback();
  } catch (error) {
    Logger.log(error);
    Logger.log(error.stack);
    return {
      status: 'error',
      message: error.toString()
    };
  } finally {
    lock.releaseLock();
  }
}

function testStartLiveAttempt() {
  Logger.log(JSON.stringify(startLiveAttempt({
    studentName: 'Test Student',
```

```
    totalQuestions: 70
  }), null, 2));
}
```